



Lecture 8: Las Vegas Algorithms, Introduction to Approximation Algorithms

Las Vegas Algorithms

- When defining the class RP, the calculations of an algorithm depend on random variables. The running time is always polynomial, the output can be incorrect with a certain probability.
- Now we consider randomized decision algorithms that do not make a false statement for languages $L \subseteq \Sigma^*$, i.e. neither in the case $w \in L$ nor in the case $w \notin L$ for all $w \in \Sigma^*$.
- However, we do not always get a reasonable, i.e. polynomial time an answer - what do we want to note with the output "??".
- We consider the running time $time_A$ of a randomized algorithm A as a random variable. We are interested in the probability distribution over the running time of words of length n ($\leq n$)

Expected Polynomial (EP):

Definition: Let Σ be an alphabet. $L \subseteq \Sigma^*$ belongs to the class EP iff there is a randomized decision algorithm A for L with a polynomial worst expected running time. Such algorithms are called Las Vegas algorithms.

Density function: $time_A(w) \mid |w| = n (\leq n)$

$$\sup\{E(time_A(w)) \mid |w| \leq n\} = O(p(n)) \text{ for a polynom } p$$

Zero-Error Probabilistic Polynomial (ZPP):

ZPP algorithms are also Las Vegas algorithms that do not make false statements, but that can also stop without making a decision - with the output "?" ("No idea").

Definition: $\varepsilon: \mathbb{N} \rightarrow [0,1)$, $L \subseteq \Sigma^*$, $L \in ZPP(\varepsilon(n))$ iff there is a probabilistic algorithm A with: $w \in \Sigma^*$

$$(1) Prob_A[A \text{ accepts } w \mid w \notin L] = 0$$

$$(2) Prob_A[A \text{ doesn't accept } w \mid w \in L] = 0$$



(3) $Prob_A[A \text{ outputs "?"}] < \varepsilon(n)$

(4) $\sup\{E(\text{time}_A(w)) \mid |w| \leq n\} = O(p(n))$ for a polynomial p

Approximation Algorithms

Definition: Let P be a problem, then:

1) A_P : Problem instances

undirected Graphs $G = (V, E)$

$V = \{1, 2, 3, \dots, n\}, n \geq 0$

$E \subseteq V \times V \quad \{x, y\} \in E, x \rightarrow y$

with $l: A_P \rightarrow \mathbb{N}_0 \quad l(V, E) = |V|$

2) $S_P(a)$: Set of valid solutions for $a \in A_P$

3) $m_P^a: S_P(a) \rightarrow \mathbb{N}_0$

4) Now we can distinguish between 2 types of optimization problems, namely a minimization problem and a maximization problem.

If P is a minimization problem, then a solution $x \in S_P(a)$ with $m_P^a(x) = \min\{m_P^a(y) \mid y \in S_P(a)\}$ should be found for $a \in A_P$.

If P is a maximization problem, then a solution $x \in S_P(a)$ with $m_P^a(x) = \max\{m_P^a(y) \mid y \in S_P(a)\}$ should be found for $a \in A_P$.

Example: P : k -colorable graph (kCG) coloring with the constraint that no adjacent nodes have the same color. Determine the minimum number of colors!

1) A_P is the set of all undirected graphs $G = (V, E)$ as well $l(G) = |V|$.

2) $S_P(G)$ is the set of all valid coloring solutions f of G .

3) $m_P^G(f)$ is the number of colors that f assigns to the nodes.

4) It is a minimization problem.



ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Quality criteria:

Let $A(a)$ be the value of the solution that Algorithm A delivers for problem instance a . Furthermore, let $OPT(a)$ be the value of the optimal solution $s \in S_P(a)$.

Absolute Error: $|OPT(a) - A(a)|$

Relative Error: $\frac{|OPT(a) - A(a)|}{OPT(a)}$

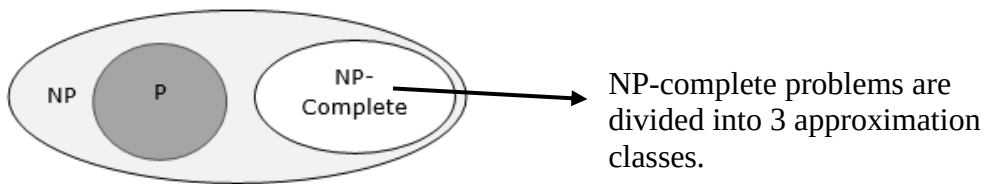
If there is an $\varepsilon > 0$ with

$$\frac{|OPT(a) - A(a)|}{OPT(a)} \leq \varepsilon, \quad \varepsilon \geq 0$$

for all $a \in A_P$, then A is called **ε -approximation** for P .

➔ There exist 3 classes for a problem $P \in NPC$ in terms of approximation.

- (1) **Fully approximable**, i.e. there is a polynomial ε -approximation for P for all $\varepsilon > 0$
- (2) **Partially approximable**, i.e. polynomial ε -approximation is only possible for particular ε -values
- (3) **Not approximable**, i.e. no polynomial ε -approximation is possible $\Rightarrow \nexists \varepsilon$



Induced Subgraph (G_{IS}):

$$G = (V, E)$$

$$U \subseteq V, E_U \subseteq E$$

$G_{IS} = (U, E_U)$ is a subgraph induced by U with $E_U = E \cap U \times U$

Complete graph with n nodes (K_n):

$$K_n = (\{1, \dots, n\}, \{1, \dots, n\} \times \{1, \dots, n\})$$



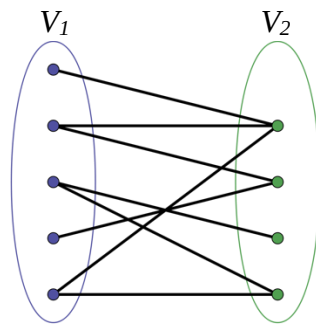
ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Bipartite Graph:

$$V_1, V_2 \subset V$$

$$E \subseteq V_1 \times V_2$$

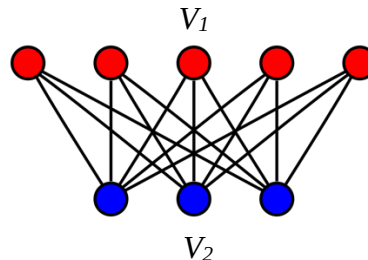
$$G = (V_1 \text{ and } V_2, E) \text{ with } V_1 \cap V_2 = \emptyset$$



Example of a bipartite graph

Complete Bipartite Graph ($K_{n,m}$):

$$V_1 = \{1, \dots, n\}, V_2 = \{1, \dots, m\}, E = V_1 \times V_2$$



Example of a complete bipartite graph with $n = 5$ and $m = 3$

Circle Graph (C_n):

$$C_n = (\{1, \dots, n\}, E)$$

$$E = \{(i, i + 1) \mid 1 \leq i \leq n - 1\} \cup \{(n, 1)\}$$

Clique Number of a Graph $\omega(G)$:

$$G = (V, E), U \subseteq V \text{ is clique iff } G_{|U} \text{ is complete.}$$

$$\text{Clique number of } G: \omega(G) = \max\{|U| \mid U \text{ is clique of } G\}$$



ÇUKUROVA UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

Independence Number of a Graph $\alpha(G)$:

$G = (U, E)$, $U \subseteq V$ is independent iff $\forall i, j \in U, i \neq j: (i, j) \notin E$

Independence number of G : $\alpha(G) = \max\{|U| \mid U \text{ is independent in } G\}$

Clique Number	Independence Number
$\omega(K_n) = n$	$\alpha(K_n) = 1$
$\omega(C_n) = 2, n = 2 \text{ and } n \geq 4$ $\omega(C_n) = 3, n = 3$	$\alpha(C_n) = \lfloor \frac{n}{2} \rfloor$
$\omega(K_{n,m}) = 2, m, n \geq 1$	$\alpha(K_{n,m}) = \max\{m, n\}$
$\alpha(G) = \omega(\bar{G})$	

Graph Coloring

$G = (V, E)$ is k -colorable iff there is a mapping $c: V \rightarrow \{1, \dots, k\}$ with $(i, j) \in E$ and $c(i) \neq c(j)$

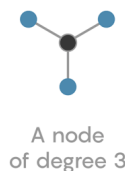
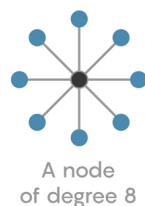
$$\chi(G) = \min\{k \mid G \text{ is } k\text{-colourable}\}$$

$\Rightarrow \chi(G)$ is the chromatic number of G

(1) $\chi(G) = 1$, if $E = \emptyset, |V| \geq 1$

(2) $\chi(G) \leq d_G + 1$

- a. The degree of a node is the number of edges that are incident on the node.
- b. Examples:



(3) $\chi(G) \geq \omega(G)$

(4) $\chi(K_n) = n$

(5) G bipartite: $\chi(G) \leq 2$

(6) $\chi(C_n) = \begin{cases} 2, & n \text{ even} \\ 3, & n \text{ odd} \end{cases}$